

17.1-AI-ASS

2303A52249

A.VAMSHI KRISHNA

Task1:

Prompt: Use AI to generate a Python script for cleaning a customer dataset.

Instructions:

Handle missing values in columns (age, income, city).

Remove duplicate records.

Standardize text values (e.g., city names in lowercase).

Encode categorical variables (gender, city).

Code and Output:

The screenshot displays a Google Colab notebook titled "17.1-AI-ASS-2249" with a file named "shopping_trends.csv". The notebook is divided into two main sections, each with a Gemini AI chat window on the right.

Section 1: Initial Data Loading

```
import pandas as pd
data = pd.read_csv("/shopping_trends.csv")
print(data.head())
```

The Gemini AI chat window on the right provides instructions for cleaning the dataset: "Use AI to generate a Python script for cleaning a customer dataset. Instructions: • Handle missing values in columns (age, income, city). • Remove duplicate records. • Standardize text values (e.g., city names in lowercase). • Encode categorical variables (gender, city)."

Section 2: Data Cleaning Steps

```
# 3. Standardize text values (e.g., City names to lowercase - assuming 'location' column acts as 'city')
if 'location' in data.columns:
    data['location'] = data['location'].str.lower()
print("Location column values converted to lowercase.")

# Display some unique values to verify
print("\nUnique values in location after standardization:")
print(data['location'].unique()[0:5]) # Displaying first 5 unique values

# 4. Encode categorical variables
# One-hot encode 'gender' and 'location'
data = pd.get_dummies(data, columns=['gender', 'location'], drop_first=True)
print("\nDataFrame head after encoding categorical variables:")
display(data.head())
```

The Gemini AI chat window on the right summarizes the progress: "I will generate a Python script to clean your customer dataset by handling missing values, removing duplicate records, standardizing text values, and encoding categorical variables. The data DataFrame is now clean and prepared for further analysis. All the requested data cleaning steps have been completed and summarized. Your data DataFrame is now clean and ready for further analysis. Is there anything else I can help you with?"

The final output shows a DataFrame with 5 rows and 67 columns, including 'Customer ID', 'Age', 'Purchased', 'Category', 'Purchase Amount (USD)', 'Size', 'Color', 'Season', 'Review Rating', 'Subscription Status', and one-hot encoded location variables (Location_south_dakota, Location_tennessee, Location_texas, Location_utah).

Observation:

Customer Data Cleaning:

Missing values and duplicates were handled to improve data quality. Text values were standardized, and categorical variables were encoded. The dataset became clean and ready for analysis.

Task2:

Prompt: Preprocess sales transaction data using AI assistance.

Instructions:

Convert date columns into datetime format.

Create new features (month, year, day of week).

Normalize sales amount column.

Detect and handle outliers in dataset.

Code and Output:

The image displays three sequential screenshots of a Google Colab notebook titled "Task-2". The notebook is running Python 3 and shows the following steps:

Step 1: Import pandas and load the data.

```
import pandas as pd
data = pd.read_csv("/sales_data.csv")
print(data.head())
```

The output shows the first five rows of the dataset:

	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold
0	1052	2023-02-03	Bob	North	5053.97	18
1	1093	2023-04-21	Bob	West	4384.02	17
2	1015	2023-09-21	David	South	4631.23	30
3	1072	2023-08-24	Bob	South	2167.94	39
4	1061	2023-03-24	Charlie	East	3750.20	13

Step 2: Data Preprocessing Steps for Sales Transaction Data.

The notebook shows the conversion of the 'Sale_Date' column to a datetime format and the creation of new features: 'Sale_Month', 'Sale_Year', and 'Sale_DayOfWeek'.

```
data['Sale_Date'] = pd.to_datetime(data['Sale_Date'])
print("Converted 'Sale_Date' to datetime format.")

# 2. Create new features: month, year, day of week
data['Sale_Month'] = data['Sale_Date'].dt.month
data['Sale_Year'] = data['Sale_Date'].dt.year
data['Sale_DayOfWeek'] = data['Sale_Date'].dt.dayofweek # Monday=0, Sunday=6
print("Created 'Sale_Month', 'Sale_Year', and 'Sale_DayOfWeek' features.")
display(data.head())
```

The output shows the first five rows of the dataset with the new features added:

	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	Product_Category	Unit_Cost	Unit_Price
0	1052	2023-02-03	Bob	North	5053.97	18	Furniture	152.75	267.22
1	1093	2023-04-21	Bob	West	4384.02	17	Furniture	3816.39	4209.44
2	1015	2023-09-21	David	South	4631.23	30	Food	261.56	371.40
3	1072	2023-08-24	Bob	South	2167.94	39	Clothing	4330.03	4467.75
4	1061	2023-03-24	Charlie	East	3750.20	13	Electronics	637.37	692.71

Step 3: Normalizing the Sales Amount column.

```
scaler = MinMaxScaler()
data['Sales_Amount_Normalized'] = scaler.fit_transform(data[['Sales_Amount']])
print("Normalized 'Sales_Amount' column to 'Sales_Amount_Normalized'.")
display(data.head())
```

The output shows the first five rows of the dataset with the 'Sales_Amount' column normalized:

	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	Product_Category	Unit_Cost	Unit_Price
0	1052	2023-02-03	Bob	North	5053.97	18	Furniture	152.75	267.22
1	1093	2023-04-21	Bob	West	4384.02	17	Furniture	3816.39	4209.44
2	1015	2023-09-21	David	South	4631.23	30	Food	261.56	371.40
3	1072	2023-08-24	Bob	South	2167.94	39	Clothing	4330.03	4467.75
4	1061	2023-03-24	Charlie	East	3750.20	13	Electronics	637.37	692.71

Step 4: Detect and handle outliers in numerical columns using IQR.

```
# 4. Detect and handle outliers in numerical columns using IQR
def cap_outliers_iqr(df, column):
    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)
```

```
# Adjust the list of columns as needed based on the dataset's actual numerical features
numerical_cols_for_outliers = ['Sales_Amount', 'Quantity_Sold', 'Unit_Price', 'Discount']

for col in numerical_cols_for_outliers:
    if col in data.columns:
        data = cap_outliers_iqr(data.copy(), col) # operate on a copy to avoid SettingWithCopyWarning
        print(f"Outliers handled for column: {col}")

print("\nDataFrame head after handling outliers:")
display(data.head())
```

Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	Product_Category	Unit_Cost	Unit_Price	Cus
0	1052	2023-02-03	Bob North	5053.97	18	Furniture	152.75	267.22	
1	1093	2023-04-21	Bob West	4384.02	17	Furniture	3816.39	4209.44	
2	1015	2023-09-21	David South	4631.23	30	Food	261.56	371.40	
3	1072	2023-08-24	Bob South	2167.94	39	Clothing	4330.03	4467.76	

Observation:

Sales Data Preprocessing:

Date features were extracted after converting to datetime format. Sales values were normalized, and outliers were handled. This improved data consistency and usability.

Task3:

Prompt: Clean and preprocess a healthcare dataset using AI scripts.

Instructions:

Handle missing values in blood_pressure, cholesterol.

Encode categorical features

Scale numerical features (min-max or standard scaling).

Split dataset into training and testing sets.

Code and Output:

```
import pandas as pd
data1 = pd.read_csv("../dirty_v2_path.csv")
print(data1.head())
```

	Age	Gender	Medical Condition	diabetes	Blood Pressure	BMI
0	40.0	Male	Diabetes	137.04	135.27	26.90
1	22.0	Male	Multihy	71.90	132.27	20.29
2	50.0	Male	Asthma	95.24	NaN	22.53
3	57.0	Male	Obesity	NaN	130.53	38.47
4	66.0	Female	Hypertension	95.19	119.17	31.12

```
# 1. Handle missing values

# Identify numerical and categorical columns for data1
numerical_cols = ['Age', 'Glucose', 'Blood Pressure', 'BMI', 'Oxygen Saturation', 'LengthOfStay', 'Cholesterol', 'Triglycerides', 'HbA1c', 'Physical Activity', 'Diet Score']
```

The screenshot shows a Google Colab notebook with the following code and output:

```

if 'heart' in data.columns:
    detail.drop(columns=['noise_col'], inplace=True)
    print("Dropped 'noise_col'.")

# Check for any remaining missing values
print('Missing values after handling:')
print(detail.isnull().sum()[detail.isnull().sum() > 0])
display(detail.head())

```

The output shows the dataset structure and the first few rows of data:

	Age	Gender	Medical Condition	Glucose	Blood Pressure	BMI	Oxygen Saturation	Length of Stay	Cholesterol	Triglycerides	HbA1c	Smoking	Alcohol	Physical Activity Score	Diet Score	Family History	Stress Level	Sleep Hours
0	45	Male	Diabetes	137.04	135.27	28.90	96.04	6	231.68	210.56	7.61	0	0	0.20	3.54	0	5.07	6.0
1	22	Male	Healthy	71.58	113.27	26.29	97.54	2	165.57	129.41	4.91	0	0	0.12	5.90	0	6.87	7.7
2	50	Female	Asthma	95.24	138.32	22.63	90.31	2	214.84	165.35	5.60	0	0	0.91	4.65	1	3.06	4.8
3	57	Female	Obesity	110.50	130.53	30.47	96.60	5	197.71	182.13	6.92	0	0	0.16	3.37	0	3.01	5.3
4	60	Female	Hypertension	95.15	178.17	31.12	94.90	4	259.53	115.85	5.98	0	1	3.56	4.40	0	6.36	6.6

Code and Output:

Lab Assignment 17.1

17.1-AI-ASS-2249 - Colab

Employee Salary

colab.research.google.com/drive/1S8ptO_4XRQze3hv3oAaVmMstOxQnn-z#scrollTo=f76afcc4

17.1-AI-ASS-2249

File Edit View Insert Runtime Tools Help

Commands + Code + Text + Run all

Files

lib64, libx32, media, mnt, opt, proc, python-apt, root, run,/sbin, srv, sys, tmp, tools, usr, var

86.51 GB available

TASK-4

```

import pandas as pd
data2 = pd.read_csv("../Employee_Salary_Dataset.csv")
print(data2.head())

```

ID	Experience_Years	Age	Gender	Salary	
0	1	5	28	Female	250000
1	2	1	21	Male	50000
2	3	3	23	Female	170000
3	4	2	22	Male	25000
4	5	1	17	Male	10000

```

import pandas as pd
from sklearn.preprocessing import LabelEncoder

print('Initial DataFrame info:')
data2.info()

print('\nMissing values before handling:')
print(data2.isnull().sum()[data2.isnull().sum() > 0])

# 1. Handle missing values
# For numerical columns, fill with median
numerical_cols = ['Experience_Years', 'Age', 'Salary']
for col in numerical_cols:
    if col in data2.columns:
        data2[col] = data2[col].fillna(data2[col].median())

# For categorical columns, fill with mode

```

Variables

Terminal

2:08 PM Python 3

Lab Assignment 17.1

17.1-AI-ASS-2249 - Colab

Employee Salary

colab.research.google.com/drive/1S8ptO_4XRQze3hv3oAaVmMstOxQnn-z#scrollTo=f76afcc4

17.1-AI-ASS-2249

File Edit View Insert Runtime Tools Help

Commands + Code + Text + Run all

Files

lib64, libx32, media, mnt, opt, proc, python-apt, root, run,/sbin, srv, sys, tmp, tools, usr, var

86.51 GB available

```

print('\nMissing values after handling:')
print(data2.isnull().sum()[data2.isnull().sum() > 0])

display(data2.head())

```

```

Initial DataFrame info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 35 entries, 0 to 34
Data columns (total 5 columns):
# Column      Non-Null Count  Dtype
---  ---
0 ID           35 non-null    int64
1 Experience_Years  35 non-null    int64
2 Age          35 non-null    int64
3 Gender       35 non-null    object
4 Salary       35 non-null    int64
dtypes: int64(4), object(1)
memory usage: 1.5+ KB

Missing values before handling:
Series([], dtype: int64)

Missing values after handling:
Series([], dtype: int64)

```

ID	Experience_Years	Age	Gender	Salary	
0	1	5	28	Female	250000
1	2	1	21	Male	50000
2	3	3	23	Female	170000
3	4	2	22	Male	25000
4	5	1	17	Male	10000

Variables

Terminal

2:08 PM Python 3

Lab Assignment 17.1

17.1-AI-ASS-2249 - Colab

Employee Salary

colab.research.google.com/drive/1S8ptO_4XRQze3hv3oAaVmMstOxQnn-z#scrollTo=f76afcc4

17.1-AI-ASS-2249

File Edit View Insert Runtime Tools Help

Commands + Code + Text + Run all

Files

lib64, libx32, media, mnt, opt, proc, python-apt, root, run,/sbin, srv, sys, tmp, tools, usr, var

86.51 GB available

```

# 2. DETECT AND REMOVE SALARY OUTLIERS USING IQR METHOD

if 'Salary' in data2.columns:
    Q1 = data2['Salary'].quantile(0.25)
    Q3 = data2['Salary'].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    initial_rows = data2.shape[0]
    data2 = data2[(data2['Salary'] >= lower_bound) & (data2['Salary'] <= upper_bound)]
    print(f'Number of salary outliers removed: {initial_rows - data2.shape[0]}')
    print(f'DataFrame shape after removing salary outliers: {data2.shape}')
else:
    print("Salary column not found in the DataFrame.")

display(data2.head())

```

```

Number of salary outliers removed: 2
DataFrame shape after removing salary outliers: (33, 5)

```

ID	Experience_Years	Age	Gender	Salary	
0	1	5	28	Female	250000
1	2	1	21	Male	50000
2	3	3	23	Female	170000
3	4	2	22	Male	25000
4	5	1	17	Male	10000

Variables

Terminal

2:08 PM Python 3

The screenshot shows a Google Colab notebook interface. The top bar indicates the notebook is titled "Employee Salary" and is part of a lab assignment. The left sidebar shows the file explorer with various folders like "lib64", "media", "mnt", "opt", "proc", "python-apt", "root", "run", "sbin", "srv", "sys", "tmp", "tools", "usr", and "var". The main area displays a code cell with the following Python code:

```
for col in categorical_cols_to_encode:
    data2[col + '_Encoded'] = le.fit_transform(data2[col])
    print(f'Column {col} label encoded to {col}_Encoded.')
    data2.drop(columns=[col], inplace=True)

print("\nDataFrame head after label encoding:")
display(data2.head())
```

The output of the code shows the following DataFrame head:

```
Column 'Gender' label encoded to 'Gender_Encoded'.

DataFrame head after label encoding:
/tmp/ipykernel_4407/3068489166.py:21: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
data2[col + '_Encoded'] = le.fit_transform(data2[col])
/tmp/ipykernel_4407/3068489166.py:23: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
data2.drop(columns=[col], inplace=True)
```

ID	Experience_Years	Age	Salary	Gender_Encoded	
0	1	5	28	250000	0
1	2	1	21	50000	1
2	3	3	23	170000	0
3	4	2	22	25000	1
4	5	1	17	10000	1

The bottom status bar shows the notebook is running on Python 3, with a search bar and system icons.

Observation:

Employee Salary Data Preprocessing:

Missing values and salary outliers were handled to ensure accuracy. Text data was standardized, and categorical variables were encoded. The dataset became structured for HR analysis.